Extensión del traductor Solidity → Liquid Haskell

Soporte de mapping(address => uint) mediante la estructura MapWithBalance

Se agregó una estructura BalancedMap (en el código, MapWithBalance) que representa el mapping como una lista clave-valor con un invariante de balance (mBal == suma de todos los valores), probado y mantenido automáticamente por Liquid Haskell en cada operación.

2. Operaciones nuevas sobre mappings (BalancedMap)

Un mapping(address => uint) de Solidity se traduce a un BalancedMap Address. Leer una clave ausente da 0, igual que en Solidity. Todas las operaciones de escritura mantienen automáticamente el invariante "mBal == suma de todos los valores del mapa", que es lo que permite luego probar que el balance nunca se vuelve negativo.

Operación	Qué hace	Qué garantiza (probado por Liquid Haskell)
currentBalance	Lee el valor asociado a una clave (equivalente a balances[key] en Solidity).	Devuelve 0 si la clave no existe todavía; nunca devuelve un valor negativo.
setWB	Escribe un valor para una clave (equivalente a balances[key] = value).	Si la clave ya existía, actualiza el balance total sumando la diferencia; si no existía, la agrega. En ambos casos mBal queda consistente con la suma real de los valores.
addWB	Agrega una clave nueva con su valor inicial (usado internamente por setWB).	El balance total nuevo es el balance anterior más el valor agregado (mBal' == mBal + value).
removeWB	Quita una clave del mapa (usado internamente por setWB al reemplazar un valor existente).	El balance total nuevo es el balance anterior menos el valor de la clave quitada, y ese balance nunca es negativo (lemma_sumBids_nonneg).
getWB	Lee el valor de una clave que se sabe presente en el mapa.	El valor devuelto es siempre >= 0 y coincide con lo almacenado realmente en el mapa.

Estas cinco operaciones son las que usa el conversor para traducir las lecturas y escrituras de mappings en el código Solidity original: balances[msg.sender] se traduce a currentBalance, y balances[msg.sender] = x o balances[msg.sender] += x se traducen a setWB sobre el resultado de currentBalance.

3. Casos verificados correctamente

Se probó la herramienta con contratos que usan mapping(address => uint) para modelar balances, en dos frentes: (a) que un contrato correcto compile y verifique como SAFE, y (b) que, siguiendo la misma metodología de la tesis al introducir un error lógico deliberado el análisis lo detecte y marque el contrato como UNSAFE.

Caso	Qué se prueba	Esperado	Obtenido
1. SimpleBank	Depósito y retiro con require(balance >= amount).	**SAFE**	**SAFE**
2. SimpleBank sin require	Se elimina la validación antes de restar.	**UNSAFE**	**UNSAFE**
3. Wallet.transfer	Transferencia con validación del balance del emisor.	**SAFE**	**SAFE**

Caso	Qué se prueba	Esperado	Obtenido
4. Wallet dirección equivocada	El require valida el balance de “to” en vez de “msg.sender”.	**UNSAFE**	**UNSAFE**
5. Rewards.redeem	Canje de puntos con require(points >= amount).	**SAFE**	**SAFE**
6. Rewards sin require	Se elimina la validación antes de restar puntos.	**UNSAFE**	**UNSAFE**
7. Operador invertido	require(balance <= amount) en vez de >=.	**UNSAFE**	**UNSAFE**
8. Validación insuficiente	require(amount > 0) en vez de validar el balance.	**UNSAFE**	**UNSAFE**

Incluyo el código Solidity completo de cada uno de los ocho casos, en el mismo orden que la tabla.

1. SimpleBank (deposit / withdraw)

Resultado esperado: **SAFE**

```
// Caso OK: deposit/withdraw con require(balance >= amount) antes de restar.
// Esperado: SAFE
contract SimpleBank {
    mapping(address => uint) public balances;
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint amount) public {
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
    }
}
```



2. SimpleBank sin require en withdraw

Resultado esperado: **UNSAFE**

```
// Caso roto: se elimina la validacion de balance suficiente antes de restar.
// Esperado: UNSAFE
contract SimpleBank {
    mapping(address => uint) public balances;
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint amount) public {
        balances[msg.sender] -= amount;
    }
}
```



3. Wallet.transfer

Resultado esperado: **SAFE**

```
// Caso OK: transferencia entre dos direcciones del mismo mapping, validando
// el balance de quien envia.
// Esperado: SAFE
contract Wallet {
    mapping(address => uint) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function transfer(address to, uint amount) public {
        require(balances[msg.sender] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
```

```
        balances[to] += amount;
    }
}
```



4. Wallet.transfer con dirección equivocada

Resultado esperado: **UNSAFE**

```
// Caso roto: el require valida el balance de "to" en lugar del de
// "msg.sender", que es quien realmente pierde fondos.
// Esperado: UNSAFE
contract Wallet {
    mapping(address => uint) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function transfer(address to, uint amount) public {
        require(balances[to] >= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
        balances[to] += amount;
    }
}
```



5. Rewards.redeem

Resultado esperado: **SAFE**

```
// Caso OK: canje de puntos con require(points >= amount) antes de restar.
// Esperado: SAFE
contract Rewards {
    mapping(address => uint) public points;

    function addPoints(uint amount) public {
        require(amount <= 100, "Too many points at once");
        points[msg.sender] += amount;
    }

    function redeem(uint amount) public {
        require(points[msg.sender] >= amount, "Not enough points");
        points[msg.sender] -= amount;
    }
}
```



6. Rewards.redeem sin require

Resultado esperado: **UNSAFE**

```
// Caso roto: se elimina toda validacion antes de restar puntos.
// Esperado: UNSAFE
contract Rewards {
    mapping(address => uint) public points;

    function addPoints(uint amount) public {
        require(amount <= 100, "Too many points at once");
        points[msg.sender] += amount;
    }

    function redeem(uint amount) public {
        points[msg.sender] -= amount;
    }
}
```



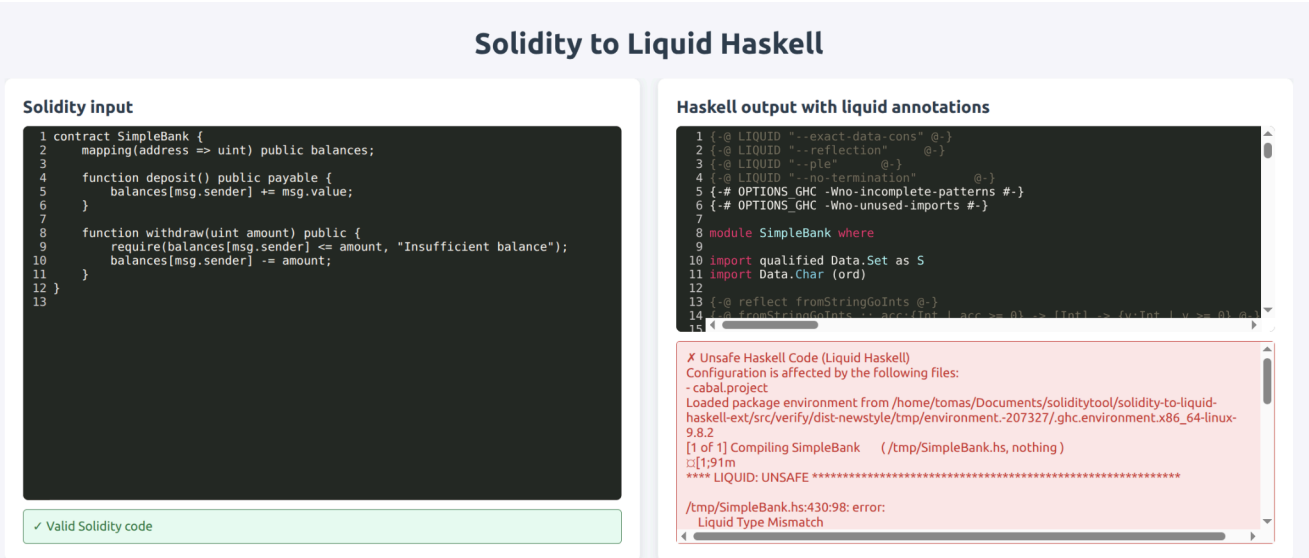
7. Operador de comparación invertido

Resultado esperado: **UNSAFE**

```
// Caso roto: require(balance <= amount) en lugar de require(balance >= amount)
// (typo tipico: operador de comparacion invertido).
// Esperado: UNSAFE
contract SimpleBank {
    mapping(address => uint) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint amount) public {
        require(balances[msg.sender] <= amount, "Insufficient balance");
        balances[msg.sender] -= amount;
    }
}
```



8. Validación insuficiente

Resultado esperado: **UNSAFE**

```
// Caso roto: valida que "amount" sea positivo, pero no que haya balance
// suficiente antes de restar.
// Esperado: UNSAFE
contract SimpleBank {
    mapping(address => uint) public balances;

    function deposit() public payable {
        balances[msg.sender] += msg.value;
    }

    function withdraw(uint amount) public {
        require(amount > 0, "Amount must be positive");
        balances[msg.sender] -= amount;
    }
}
```



4. Cambios más grandes sobre el traductor base

Además de la funcionalidad nueva (BalancedMap), agregar soporte de mappings obligó a modificar una decisión de diseño que ya existía en el traductor base de la tesis: qué tipo de Haskell usa cada tipo numérico de Solidity.

Unificación de Uint / Amount / Address en Int

La librería BalancedMap incorporada (adaptada de MapWithBalance.hs, ya demostrada de antes) usa Int como tipo de los valores del mapa, no Integer. Esto no es una preferencia de estilo: usar Integer ahí rompe la prueba de la propiedad lemma_sumBids_nonneg (“el balance total nunca es negativo”). La razón técnica es que la verificación de Liquid Haskell con --ple (“Proof by Logical Evaluation”) necesita poder desplegar la definición del tipo paso a paso durante la prueba, y el Integer de Haskell (de precisión arbitraria) está construido de una forma que no se despliega igual que Int y bloquea esa prueba.

Como los balances de un contrato normalmente se usan mezclados —una función puede sumar el resultado de currentBalance (un valor que viene del mapping) contra otra variable de estado numérica común, como balance o un contador—, hacía falta que todos esos tipos numéricos “hablaran el mismo idioma”. Por eso, en typeMappings.js, los tres tipos que un mapping puede necesitar tocar quedaron unificados en Int:

```
type Amount = Int    -- balance total del contrato
type Uint = Int      -- uint / uint256 de Solidity
```

```
type Address = Int   -- address de Solidity (representado como un numero)
```

Antes de este cambio, un uint256 de Solidity se traducía directamente a Integer (igual que un int con signo). Ahora uint, address y el balance del contrato pasan a Int, que es un entero de precisión fija pero acotado con un refinamiento de Liquid Haskell (v >= 0), y sigue comportándose como un número no negativo en la práctica.

4. Limitaciones conocidas

Además de las limitaciones ya documentadas en la tesis original (arrays, loops, structs, eventos y llamadas a funciones no view/pure quedan fuera de alcance), esta extensión agrega las siguientes:

Limitación	Detalle
Postcondiciones sobre variables locales	Un assert que compara el estado actual contra una variable local que guardó un valor anterior del mapping (patrón "uint before = balances[msg.sender]; ... assert(balances[msg.sender] == before + amount)") no se traduce correctamente: la anotación de tipo generada no puede referirse a esa variable local (solo puede hablar de los parámetros de entrada y del valor de retorno), y el conversor actualmente emite un placeholder inválido. Esto produce un error de compilación ("Illegal type specification"), no una falla real de la prueba. Los postcondiciones que sí funcionan son las que comparan directamente contra variables de estado o parámetros, igual que en los ejemplos de la tesis original.
Un único mapping por contrato probado	Los casos de prueba usan un solo mapping(address => uint) por contrato. No se probó todavía la interacción entre dos o más mappings distintos dentro del mismo contrato.
Solo mapping(address => uint)	No se soportan mappings a otros tipos de valor (structs, enums, otros mappings anidados) ni mappings con clave distinta de address.